

Algorytm AES - projekt

Aleksandra Gołek | 151429 | informatyka

Spis treści

Temat.....	2
Opis matematyczny analizowanego algorytmu	3
1. SubBytes – zamiana bajtów według tabeli S-box	3
2. ShiftRows – przesuwanie kolumn.....	5
3. MixColumns – mieszanie kolumn.....	6
4. AddRoundKey – XOR za pomocą klucza rundowego.....	7
Pseudokod algorytmu.....	9
Szyfrowanie AES.....	9
Deszyfrowanie AES	9
Dodatkowe funkcje pomocnicze.....	9
Krótką instrukcja obsługi	11
1. Wprowadzenie tekstu.....	11
2. Szyfrowanie.....	11
3. Deszyfrowanie	11
Wskazówki	12
Kod w Python.....	13
Kod w Python z użyciem biblioteki cryptography	14
Źródła.....	15

Temat

Celem projektu jest przedstawienie, jak działa algorytm AES (Advanced Encryption Standard), poprzez implementację jego funkcjonalności szyfrowania i deszyfrowania.

W programie użytkownik podaje tekst do zaszyfrowania oraz tajny klucz, a wynik jest prezentowany w formie ciągu zakodowanych znaków (szyfrogramu). Program umożliwia także odszyfrowanie tekstu, jeśli podano poprawny klucz.

W sprawozdaniu skupimy się na algorytmie dla klucza 128 bitowego. Dodatkowo, w języku Python zaimplementowałam prosty algorytm AES bez użycia dostępnych bibliotek do kryptografii, dzięki czemu łatwiej jest wyjaśnić działanie tego algorytmu.

Opis matematyczny analizowanego algorytmu

AES to algorytm szyfrujący blokowy, który działa na blokach danych o stałej wielkości (128 bitów, tj. 16 bajtów). Dane są przekształcane w serii rund, które składają się z czterech podstawowych operacji matematycznych.

Etapy szyfrowania AES:

1. SubBytes – zamiana bajtów według tabeli S-box

Każdy bajt bloku wejściowego jest zastępowany przez nową wartość według tabeli S-box (tabela substytucji). Operacja ta wprowadza nieliniowość w algorytmie.

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xa	xb	xc	xd	xe	xf
0x	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1x	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2x	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3x	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4x	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5x	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6x	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7x	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8x	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9x	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
ax	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
bx	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
cx	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
dx	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
ex	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
fx	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

$$b' = \text{S-Box}[b]$$

S-box jest generowany przy użyciu:

- Odwrotności modularnej bajtu w ciele skończonym $\text{GF}(2^8)$, gdzie operacje są wykonywane z użyciem wielomianu redukującego

$$x^8 + x^4 + x^3 + x + 1$$

Odwrotność modularną bajtu wyznacza się za pomocą działania modulo.

$$b \cdot b^{-1} \mod p(x) = 1$$

- Transformacji afinicznej, która wprowadza nieliniowość:

$$S[b] = A \cdot b^{-1} \oplus c$$

- A to stała macierz 8x8.
- b^{-1} to odwrotność modularna bajtu b.
- c to stała wektorowa.

Przykład obliczania wartości S-box dla litery:

Litera „a” (w ASCII) ma kod szesnastkowy 0x61 i binarną reprezentację 011000012011000012011000012. Aby znaleźć jej wartość w S-boxie:

1. Zaczynamy od przekształcenia litery na jej wartość w systemie szesnastkowym:
 $a = 0x61 = 01100001_2$
2. Obliczenia dla S-boxa. S-box jest zbiorem wartości przeliczeniowych, a każda pozycja odpowiada danej wartości wejściowej (w naszym przypadku 0x61).

2.1 Wyznaczenie modularnej odwrotności a^{-1}

W $GF(2^8)$, znajdujemy a^{-1} taki że:

$$a \cdot a^{-1} \mod p(x) = 1$$

dla $p(x) = x^8 + x^4 + x^3 + x + 1$

Dla $a = 0x61$, obliczenia (przy pomocy tabel precomputing lub algorytmu rozszerzonego Euklidesa) dają:

$$a^{-1} = 0x45 = 01000101_2$$

2.2 Transformacja afiniczna (definicja powyżej)

Przy ustalonej macierzy A (8x8) i wektorze c (8-bitowy), wykonujemy mnożenie macierzy (mod 2) i operację XOR.

Po obliczeniach litera „a” (0x61) w S-box jest przekształcona w 0xE2.

Przykład odczytania wartości z S-box:

Jeśli bajt wejściowy to 0x53 (w zapisie szesnastkowym):

$$b' = S\text{-Box}[0x53] = 0xed$$

- Bajt 0x53 wskazuje na wiersz 5 i kolumnę 3 w tabeli S-box.
- Wartość w tej pozycji, np. 0xED, zastępuje 0x53.

W poniżej przedstawionym programie używamy listy z wartościami S-box (w celu zaszyfrowania), a także tabeli do niej odwrotnej (do odszyfrowania).

2. ShiftRows – przesuwanie kolumn

Macierz danych jest interpretowana jako tablica 4x4, gdzie wiersze są przesuwane w następujący sposób:

$$\text{Macierz wejściowa: } \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \\ b_{20} & b_{21} & b_{22} & b_{23} \\ b_{30} & b_{31} & b_{32} & b_{33} \end{bmatrix}$$

$$\text{Po przesunięciu: } \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{11} & b_{12} & b_{13} & b_{10} \\ b_{22} & b_{23} & b_{20} & b_{21} \\ b_{33} & b_{30} & b_{31} & b_{32} \end{bmatrix}$$

Wiersze macierzystą przesuwane o różne liczby pozycji w lewo:

- Wiersz 1: Bez zmian.
- Wiersz 2: Przesunięcie o 1 miejsce.
- Wiersz 3: Przesunięcie o 2 miejsca.
- Wiersz 4: Przesunięcie o 3 miejsca.

$$\begin{bmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{bmatrix}$$

Przykład:

Macierz przed i po operacji ShiftRows (rysunek obok).

$$\begin{bmatrix} A & B & C & D \\ F & G & H & E \\ K & L & I & J \\ P & M & N & O \end{bmatrix}$$

3. MixColumns – mieszanie kolumn

Kolumny macierzy danych są mieszane poprzez mnożenie przez stałą macierz w ciele skończonym $GF(2^8)$.

$$\begin{bmatrix} c'_0 \\ c'_1 \\ c'_2 \\ c'_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

- MixColumns traktuje każdą kolumnę macierzy state jako wektor czteroelementowy.
- Każda kolumna jest mnożona przez stałą macierz w ciele skończonym $GF(2^8)$.

$$M = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix}$$

Matematycznie:

- Mnożenie odbywa się modulo wielomian $x^8 + x^4 + x^3 + x + 1$.
- Wynikowa kolumna to:

$$C'[i] = M \cdot C[i]$$

gdzie $C[i]$ to kolumna i w macierzy.

Przykład:

Dla kolumny $C = [a, b, c, d]$:

- Nowa kolumna to:

$$\begin{bmatrix} 2a + 3b + c + d \\ a + 2b + 3c + d \\ a + b + 2c + 3d \\ 3a + b + c + 2d \end{bmatrix}$$

Mnożenie i dodawanie odbywa się modulo 256.

4. AddRoundKey – XOR za pomocą klucza rundowego

Każdy bajt danych jest XOR-owany z odpowiadającym bajtem klucza rundowego, który powstał w wyniku ekspansji klucza wejściowego.

- AddRoundKey to operacja XOR pomiędzy macierzą state (stan - macierz 4x4 bajtów (16 bajtów danych), reprezentująca aktualny stan bloku danych) a kluczem rundowym (macierz 4x4 bajtów (16 bajtów), która jest częścią klucza uzyskaną w procesie Key Expansion).
- Klucz rundowy jest generowany z klucza głównego w procesie Key Expansion.

Każdy bajt w stanie jest modyfikowany:

$$S[i][j] = S[i][j] \oplus K[i][j]$$

gdzie $S[i][j]$ to bajt w stanie, a $K[i][j]$ to bajt w odpowiadającej pozycji klucza rundowego.

$$b'_i = b_i \oplus k_i$$

Jeśli bajt $S[0][0] = 0x3C$, a bajt klucza $K[0][0] = 0xA1$:

$$0x3C \oplus 0xA1 = 0x9D$$

Przykład:

Wejściowy State - Macierz danych (np. w zapisie szesnastkowym):

$$\text{State} = \begin{bmatrix} 0x32 & 0x88 & 0x31 & 0xe0 \\ 0x43 & 0x5a & 0x31 & 0x37 \\ 0xf6 & 0x30 & 0x98 & 0x07 \\ 0xa8 & 0x8d & 0xa2 & 0x34 \end{bmatrix}$$

RoundKey - Macierz klucza rundowego:

$$\text{RoundKey} = \begin{bmatrix} 0x2b & 0x28 & 0xab & 0x09 \\ 0x7e & 0xae & 0xf7 & 0xcf \\ 0x15 & 0xd2 & 0x15 & 0x4f \\ 0x16 & 0xa6 & 0x88 & 0x3c \end{bmatrix}$$

XOR: Każdy bajt w macierzy State jest XOR-owany z odpowiadającym mu bajtem z RoundKey.

1. Pierwszy wiersz:

$$0x32 \oplus 0x2b = 0x19, \quad 0x88 \oplus 0x28 = 0xa0, \quad 0x31 \oplus 0xab = 0x9a, \quad 0xe0 \oplus 0x09 = 0xe9$$

2. Drugi wiersz:

$$0x43 \oplus 0x7e = 0x3d, \quad 0x5a \oplus 0xae = 0xf4, \quad 0x31 \oplus 0xf7 = 0xc6, \quad 0x37 \oplus 0xcf = 0xf8$$

3. Trzeci wiersz:

$$0xf6 \oplus 0x15 = 0xe3, \quad 0x30 \oplus 0xd2 = 0xe2, \quad 0x98 \oplus 0x15 = 0x8d, \quad 0x07 \oplus 0x4f = 0x48$$

4. Czwarty wiersz:

$$0xa8 \oplus 0x16 = 0xbe, \quad 0x8d \oplus 0xa6 = 0x2b, \quad 0xa2 \oplus 0x88 = 0x2a, \quad 0x34 \oplus 0x3c = 0x08$$

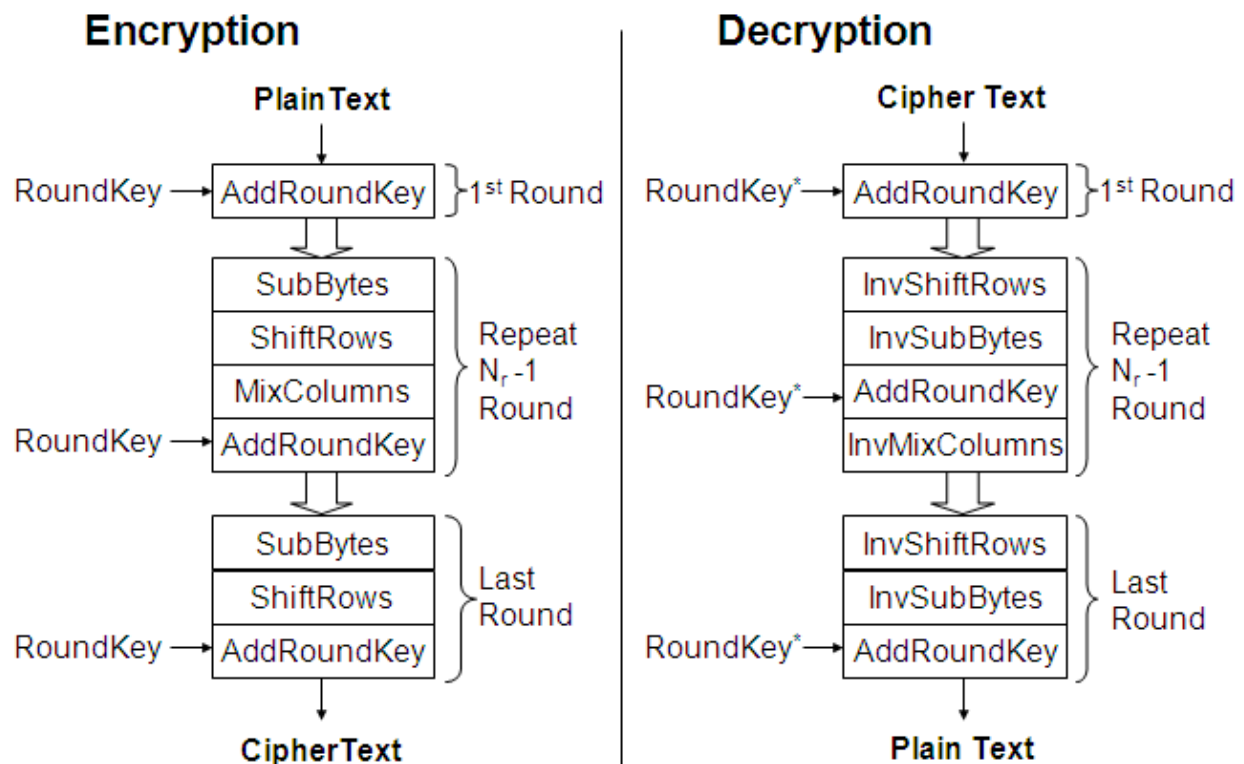
Po operacji AddRoundKey macierz State wygląda następująco:

$$\text{State} = \begin{bmatrix} 0x19 & 0xa0 & 0x9a & 0xe9 \\ 0x3d & 0xf4 & 0xc6 & 0xf8 \\ 0xe3 & 0xe2 & 0x8d & 0x48 \\ 0xbe & 0x2b & 0x2a & 0x08 \end{bmatrix}$$

Operacja XOR wprowadza element zależny od klucza, co czyni szyfrogram trudnym do przewidzenia bez znajomości klucza. Ta sama operacja może być użyta w deszyfrowaniu, ponieważ XOR z tym samym kluczem przywraca oryginalny stan.

Cały algorytm można przedstawić za pomocą schematu blokowego.

Szyfrowanie i deszyfrowanie AES, źródło: https://www.researchgate.net/figure/AES-Encryption-Decryption-Flowchart_fig2_221958203



Pseudokod algorytmu

Wszystkie opisane funkcje zostały zastosowane w kodzie źródłowym.

Szyfrowanie AES

Funkcja AES_Encrypt(tekst, klucz):

1. Sprawdź, czy długość klucza jest 128 bitów (16 bajtów).
2. Rozpocznij rozszerzanie klucza (Key Expansion).
3. Podziel tekst na bloki o rozmiarze 128 bitów (16 bajtów).
4. Dla każdego bloku:
 - 4.1. Przekształć blok w macierz 4x4 (state).
 - 4.2. Dodaj klucz rundy 0 do stanu (AddRoundKey).
 - 4.3. Dla rund 1 do 9: (liczba rund jest dowolna)
 - 4.3.1. Zastosuj podstawienie bajtów (SubBytes).
 - 4.3.2. Zastosuj przesunięcie wierszy (ShiftRows).
 - 4.3.3. Zastosuj mieszanie kolumn (MixColumns).
 - 4.3.4. Dodaj klucz rundy (AddRoundKey).
 - 4.4. Dla rundy 10:
 - 4.4.1. Zastosuj podstawienie bajtów (SubBytes).
 - 4.4.2. Zastosuj przesunięcie wierszy (ShiftRows).
 - 4.4.3. Dodaj klucz rundy (AddRoundKey).
 - 4.5. Zwróć zaszyfrowany blok.
5. Zwróć cały zaszyfrowany tekst (po złączeniu zaszyfrowanych bloków).

Deszyfrowanie AES

Funkcja AES_Decrypt(tekst, klucz):

1. Sprawdź, czy długość klucza jest 128 bitów (16 bajtów).
w programie używamy klucza 128 bitowego.
2. Rozpocznij rozszerzanie klucza (Key Expansion).
3. Podziel tekst na bloki o rozmiarze 128 bitów (16 bajtów).
4. Dla każdego bloku:
 - 4.1. Przekształć blok w macierz 4x4 (state).
 - 4.2. Dodaj klucz rundy 10 do stanu (AddRoundKey).
 - 4.3. Dla rund 9 do 1: (liczba rund jest taka sama jak w encryption)
 - 4.3.1. Zastosuj odwrotne przesunięcie wierszy (InvShiftRows).
 - 4.3.2. Zastosuj odwrotne podstawienie bajtów (InvSubBytes).
 - 4.3.3. Dodaj klucz rundy (AddRoundKey).
 - 4.3.4. Zastosuj odwrotne mieszanie kolumn (InvMixColumns).
 - 4.4. Dla rundy 0:
 - 4.4.1. Zastosuj odwrotne przesunięcie wierszy (InvShiftRows).
 - 4.4.2. Zastosuj odwrotne podstawienie bajtów (InvSubBytes).
 - 4.4.3. Dodaj klucz rundy (AddRoundKey).
 - 4.5. Zwróć odszyfrowany blok.
5. Zwróć cały odszyfrowany tekst (po złączeniu odszyfrowanych bloków).

Dodatkowe funkcje pomocnicze

- **Key Expansion (Rozszerzanie klucza):**

Rozszerzenie klucza odbywa się poprzez tworzenie nowych kluczy rundowych, które są wykorzystywane w każdej rundzie algorytmu. Do tworzenia nowych kluczy wykorzystuje się

różne operacje, takie jak przesunięcia, podstawienie bajtów, oraz operacja Rcon (rotacja klucza).

- **SubBytes (Podstawienie bajtów):**

Zastosowanie funkcji podstawienia bajtów na każdy bajt w stanie, używając tabeli S-box (AES S-box). Każdy bajt w stanie jest zastępowany odpowiednim bajtem z tabeli.

- **ShiftRows (Przesunięcie wierszy):**

Przesunięcie wierszy stanu w lewo, w zależności od numeru wiersza. Pierwszy wiersz pozostaje bez zmian, drugi jest przesuwany o jedno miejsce w lewo, trzeci o dwa miejsca, a czwarty o trzy miejsca.

- **MixColumns (Mieszanie kolumn):**

Operacja, która miesza dane w każdej kolumnie stanu. Mieszanie kolumn jest stosowane w rundach 1 do n, a w ostatniej rundzie jest pomijane.

- **AddRoundKey (Dodanie klucza rundy):**

Zastosowanie operacji XOR pomiędzy stanem a odpowiednim kluczem rundy. Klucz rundy jest generowany na podstawie oryginalnego klucza i kluczy pośrednich.

- **InvSubBytes (Odwrotne podstawienie bajtów):**

Odwrotność operacji SubBytes. Zamiast korzystać z S-box, używamy tabeli odwrotnej S-box (Inversed S-box), aby odwrócić podstawienie.

- **InvShiftRows (Odwrotne przesunięcie wierszy):**

Odwrócenie operacji ShiftRows. Zamiast przesuwać wiersze w lewo, przesuwamy je w prawo.

- **InvMixColumns (Odwrotne mieszanie kolumn):**

Odwrócenie operacji MixColumns. Jest stosowane w rundach deszyfrowania, ale pomijane w ostatniej rundzie.

Krótką instrukcja obsługi

1. Wprowadzenie tekstu

- Program najpierw prosi użytkownika o wprowadzenie tekstu do zaszyfrowania.

Enter your string to encrypt:

Hello, AES!

2. Szyfrowanie

- Po podaniu tekstu, program automatycznie go dopełnia do wymaganego rozmiaru (jeśli jest to konieczne) i przeprowadza szyfrowanie za pomocą funkcji `aes_encrypt_block`.

```
Original Text: Hello, AES!  
Plaintext (Padded): b'Hello, AES!\x05\x05\x05\x05\x05'
```

- Wynik szyfrowania zostanie wyświetlony w postaci ciągu znaków w formacie szesnastkowym.

```
-- Encryption --  
  
Encrypted Text: 48656c6c6f2c20414553210505050505
```

3. Deszyfrowanie

- Program następnie wykonuje deszyfrowanie przy użyciu tej samej funkcji `aes_decrypt_block` i wyświetla oryginalny tekst po odszyfrowaniu.

```
-- Decryption --  
  
Decrypted Text: Hello, AES!
```

Wskazówki

- Dopełnianie tekstu:

Jeśli wprowadzony tekst nie ma długości 16 bajtów, program automatycznie dopełnia go odpowiednimi bajtami.

- Klucz: Klucz musi być określony w formie macierzy 4x4 (16 bajtów). W programie domyślnie zdefiniowany jest przykładowy klucz:

```
key = [ [0x2b, 0x7e, 0x15, 0x16],  
        [0x28, 0xae, 0xd2, 0xa6],  
        [0xab, 0xf7, 0x97, 0x75],  
        [0x46, 0x1f, 0x3c, 0x08] ]
```

W sytuacji rzeczywistej użytkownik powinien znać klucz, którym zaszyfrowano tekst, w celu jego odszyfrowania.

- Format wejściowy i wyjściowy: Program przyjmuje tekst wejściowy w formie zwykłego ciągu znaków i zwraca tekst zaszyfrowany w formacie szesnastkowym. Po deszyfrowaniu tekst wraca do formy oryginalnej (jeśli nie był zmieniany).

Kod w Python

```
import numpy as np
```

```
# Full S-box (AES S-box)
S_BOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf9, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0x47, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x00, 0x83, 0xc2, 0x1a, 0x1b, 0x6e, 0x5a, 0xa8, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcfc,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x00, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x40, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0xdd, 0xdd, 0x05, 0x0e, 0xa9, 0xdc, 0x56, 0xf4, 0xea, 0x05, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x99, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x5c, 0xa1, 0x89, 0x0d, 0xbf, 0x6e, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x8f, 0xb0, 0x54, 0xbb, 0x16
]

# Inverse S-box (AES Inverse S-box)
INVERSE_S_BOX = [
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb,
    0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43, 0x44, 0xc4, 0xa5, 0x00, 0x54,
    0x7b, 0x79, 0x3d, 0x33, 0x80, 0x4b, 0x1b, 0x8d, 0x40, 0x46, 0xc6, 0xc, 0x56, 0x72, 0x64, 0x5f,
    0x34, 0x0c, 0x3c, 0x4a, 0x01, 0x3e, 0x21, 0x3a, 0x3d, 0x53, 0x40, 0x23, 0x1b, 0x11, 0x67, 0x77,
    0x24, 0x27, 0x7d, 0x2e, 0x8b, 0x87, 0xd9, 0x39, 0x2c, 0x42, 0x58, 0x40, 0x69, 0x9f, 0x2e, 0x1e,
    0x1c, 0x3b, 0x8c, 0x5c, 0x56, 0xe6, 0x5f, 0x79, 0x42, 0x70, 0x25, 0x45, 0x12, 0x24, 0x6b, 0x16,
    0x77, 0x11, 0x88, 0xa3, 0x33, 0x99, 0x9b, 0x80, 0xd1, 0xf4, 0xc1, 0x3a, 0x7c, 0xf0, 0x66, 0x6f,
    0x6d, 0x6b, 0x23, 0x21, 0x98, 0x3f, 0x25, 0x8e, 0x62, 0x14, 0x92, 0x13, 0xf9, 0x89, 0x3f, 0x8d
]

# AES block size (16 bytes)
BLOCK_SIZE = 16

def pad_text(text):
    padding_length = BLOCK_SIZE - len(text) % BLOCK_SIZE
    return text + bytes([padding_length] * padding_length)

def unpad_text(text):
    padding_length = text[-1]
    return text[:-padding_length]

def sub_bytes(state):
    return [[S_BOX[byte] for byte in row] for row in state]

def inv_sub_bytes(state):
    return [[INVERSE_S_BOX[byte] for byte in row] for row in state]

def shift_rows(state):
    return [
        state[0],
        state[1][1:] + state[1][:1],
        state[2][2:] + state[2][:2],
        state[3][3:] + state[3][:3]
    ]

def inv_shift_rows(state):
    return [
        state[0],
        state[1][-1:] + state[1][:-1],
        state[2][-2:] + state[2][:-2],
        state[3][-3:] + state[3][:-3]
    ]

def mix_columns(state):
    def xtime(x):
        return ((x << 1) & 0xFF) ^ (0x11B if x & 0x80 else 0)

    def mix_single_column(column):
        return [
            xtime(column[0]) ^ column[1] ^ column[2] ^ column[3],
            column[0] ^ xtime(column[1]) ^ column[2] ^ column[3],
            column[0] ^ column[1] ^ xtime(column[2]) ^ column[3],
            column[0] ^ column[1] ^ column[2] ^ xtime(column[3])
        ]

    return [mix_single_column(col) for col in zip(*state)]

def inv_mix_columns(state):
    def xtime(x):
        return ((x << 1) & 0xFF) ^ (0x11B if x & 0x80 else 0)

    def inv_mix_single_column(column):
        return [
            xtime(xtime(column[0])) ^ xtime(column[1]) ^ column[2] ^
            column[3],
            column[0] ^ xtime(xtime(column[1])) ^ xtime(column[2]) ^
            column[3],
            column[0] ^ column[1] ^ xtime(xtime(column[2])) ^
            xtime(column[3]),
            xtime(column[0]) ^ column[1] ^ column[2] ^
            xtime(xtime(column[3]))
        ]

    return [inv_mix_single_column(col) for col in zip(*state)]

def add_round_key(state, key):
    return [[state[i][j] ^ key[i][j] for j in range(4)] for i in range(4)]

def key_expansion(key):
    round_keys = [key]
    Rcon = [0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36]
    return round_keys

def aes_encrypt_block(plaintext, key):
    state = [[list(plaintext[i:i+4]) for i in range(0, len(plaintext), 4)]]
```

```
# Initial Add Round Key
add_round_key(state, key)

# Perform 4 rounds (as requested)
for round_num in range(1, 4):
    sub_bytes(state)
    shift_rows(state)
    mix_columns(state)
    add_round_key(state, key) # Pass the key here

# Final round (no mixing)
sub_bytes(state)
shift_rows(state)
add_round_key(state, key) # Pass the key here

# Convert state back to ciphertext (as bytes)
ciphertext = [byte for row in state for byte in row]
return ciphertext

def aes_decrypt_block(ciphertext, key):
    state = [[list(ciphertext[i:i+4]) for i in range(0, len(ciphertext), 4)]]

# Initial Add Round Key
add_round_key(state, key)

# Perform 4 rounds (as requested)
for round_num in range(4, 1, -1):
    inv_shift_rows(state) # Ensure this modifies the state
    inv_sub_bytes(state) # Ensure this modifies the state
    add_round_key(state, key) # Pass the key here
    inv_mix_columns(state) # Ensure this modifies the state

# Final round (no mixing)
inv_shift_rows(state)
inv_sub_bytes(state)
add_round_key(state, key) # Pass the key here

# Convert state back to plaintext
plaintext = ''.join([chr(byte) for row in state for byte in row])
return plaintext

def main():
    key = [ 0x2b, 0x7e, 0x15, 0x16], [0x28, 0xae, 0xd2, 0xa6], [0xab, 0xf7, 0x97, 0x75], [0x46, 0x1f, 0x3c, 0x08] ]
    plaintext = input("Enter your string to encrypt: ")
    plaintext_bytes = pad_text(plaintext.encode())

    print("Original Text: ", plaintext)
    print("Plaintext (Padded):", plaintext_bytes)

    print("\n-- Encryption --")
    encrypted_state = aes_encrypt_block(plaintext_bytes, key)
    encrypted_bytes = bytes(encrypted_state) # Convert encrypted state to bytes

    print("\nEncrypted Text:", encrypted_bytes.hex())

    print("\n-- Decryption --")
    decrypted_state = aes_decrypt_block(encrypted_bytes, key)

    # No need to convert decrypted_state to bytes, just unpad and decode
    decrypted_text = unpad_text(decrypted_state.encode()).decode() # Decode to text after unpadding

    print("\nDecrypted Text:", decrypted_text)

if __name__ == "__main__":
    main()
```

Kod w Python z użyciem biblioteki cryptography

Pozwala na zaszyfrowanie polskich znaków.

```
from cryptography.hazmat.primitives.ciphers
import Cipher, algorithms, modes
from cryptography.hazmat.backends import
default_backend
from cryptography.hazmat.primitives import
padding
import os

def aes_encrypt(plaintext, key):
    # Generowanie wektora inicjalizacyjnego
    (IV) dla trybu ECB
    # W trybie ECB, IV jest nieużywany, ale
    AES wymaga podania losowego IV, nawet w tym
    trybie.
    iv = os.urandom(16) # AES blok ma 16
    bajtów

    # Tworzenie obiektu Cipher
    cipher = Cipher(algorithms.AES(key),
    modes.ECB(), backend=default_backend())

    # Dodawanie paddingu (AES wymaga bloków
    o rozmiarze 16 bajtów)
    padder = padding.PKCS7(128).padder() #
    128 to długość bloku w bitach
    padded_data =
    padder.update(plaintext.encode()) +
    padder.finalize()

    # Szyfrowanie danych
    encryptor = cipher.encryptor()
    ciphertext =
    encryptor.update(padded_data) +
    encryptor.finalize()

    return ciphertext, iv

def aes_decrypt(ciphertext, key):
    # Tworzenie obiektu Cipher
    cipher = Cipher(algorithms.AES(key),
    modes.ECB(), backend=default_backend())
```

```
# Deszyfrowanie danych
decryptor = cipher.decryptor()
decrypted_data =
decryptor.update(ciphertext) +
decryptor.finalize()

# Usuwanie paddingu
unpadder =
padding.PKCS7(128).unpadder()
unpadded_data =
unpadder.update(decrypted_data) +
unpadder.finalize()

return unpadded_data.decode()

def main():
    # Klucz AES (musimy go ustawić na 16,
    24 lub 32 bajty - 128, 192 lub 256 bitów)
    key = os.urandom(16) # Losowy klucz
    16-bajtowy (128-bitowy AES)

    # Wprowadzenie tekstu przez użytkownika
    plaintext = input("Enter your string to
    encrypt (including Polish characters): ")

    print("\nOriginal Text: ", plaintext)

    # Szyfrowanie
    ciphertext, iv = aes_encrypt(plaintext,
    key)
    print("\nEncrypted Text (hex):",
    ciphertext.hex())

    # Deszyfrowanie
    decrypted_text =
    aes_decrypt(ciphertext, key)
    print("\nDecrypted Text: ",
    decrypted_text)

if __name__ == "__main__":
    main()
```

Output:

Enter your string to encrypt (including Polish characters):

Original Text: Zażółć gęsią jaźń

Encrypted Text (hex): dc0d765ac2b4d381ea9b589f9fdb8185d0dc1df9fcfd92c29a275d7fe18f7bd3

Decrypted Text: Zażółć gęsią jaźń

Źródła

- https://en.wikipedia.org/wiki/Advanced_Encryption_Standard
- <https://www.techtarget.com/searchsecurity/definition/Advanced-Encryption-Standard>
- <https://github.com/Cojacfar/AES/blob/master/AES.py>
(Elementy programu w Python; tabela S-box w formie listy)
- https://ssamolej.kia.prz.edu.pl/dydaktyka/KiBD/04_KiBD_szyfry_blokowe1.pdf
- <https://femionewin.medium.com/aes-encryption-with-python-step-by-step-3e3ab0b0fd6c>

Programy można uruchomić online za pomocą:

- https://www.onlinegdb.com/online_python_compiler#
- <https://onecompiler.com/python>